

Smart Food Delivery & Order Management System

5-Way Group Task Split (Java Implementation)

Group Member 1 — User & Restaurant Management

Module:

User and Restaurant Management System

Responsibilities:

- Create Java classes for:
 - Customer
 - Restaurant
- Implement:
 - Add user/restaurant
 - Delete user/restaurant
 - Display all users/restaurants
- Use:
 - `ArrayList` or `LinkedList`
- Explain:
 - Why the chosen data structure is suitable
 - Time complexity for add/delete/search operations

Suggested Files:

```
Customer.java
Restaurant.java
UserManager.java
RestaurantManager.java
```

Features to Demonstrate:

- Dynamic insertion/removal
- Menu-driven management system
- Data display formatting

Concepts Covered:

- Array / Linked List
 - Object-Oriented Programming
 - CRUD operations
-

Group Member 2 — Order Processing System

Module:

Real-Time Order Processing

Responsibilities:

- Implement:
 - Order Queue (FIFO)
 - Undo feature using Stack (LIFO)
- Create:
 - Add order
 - Process order
 - Undo last added item
 - Confirm order
- Use:
 - Queue
 - Stack

Suggested Files:

```
Order.java
OrderQueue.java
UndoStack.java
OrderSystem.java
```

Features to Demonstrate:

- Orders processed in arrival order
- Undo last item before confirmation
- Real-time simulation

Concepts Covered:

- Queue (FIFO)

- Stack (LIFO)
 - Java Collections Framework
-

Group Member 3 — Delivery Assignment System

Module:

Delivery Rider Priority Assignment

Responsibilities:

- Create Rider class
- Assign riders based on:
 - Shortest delivery time
 - Nearest distance
- Implement:
 - Priority Queue (Min Heap)
- Compare:
 - Priority Queue vs Linear Search

Suggested Files:

```
Rider.java  
DeliveryManager.java  
PriorityRiderQueue.java
```

Features to Demonstrate:

- Automatic best rider selection
- Priority updates
- Efficient retrieval

Concepts Covered:

- Priority Queue
 - Min Heap
 - Comparator Interface
 - Performance Optimization
-

Group Member 4 — Route Optimization System

Module:

Shortest Path Navigation

Responsibilities:

- Represent locations as Graph
- Implement:
 - Dijkstra Algorithm
- Calculate:
 - Shortest route between restaurant and customer
- Explain:
 - How Dijkstra works
 - Complexity analysis

Suggested Files:

```
Graph.java
Node.java
Edge.java
DijkstraAlgorithm.java
RouteManager.java
```

Features to Demonstrate:

- Weighted graph
- Route calculation
- Path visualization/output

Concepts Covered:

- Graph Data Structure
 - Adjacency List
 - Dijkstra Algorithm
 - Shortest Path Problem
-

Group Member 5 — Search & Data Retrieval System

Module:

Food Search + Fast Data Retrieval

Responsibilities:

Part A — Food Search & Recommendation

- Implement:
 - BST or AVL Tree
- Features:
 - Insert food items
 - Search food items
 - Sort/display foods

Part B — Data Retrieval Optimization

- Implement:
 - HashMap for:
 - User profiles
 - Order details
- Explain:
 - $O(1)$ retrieval efficiency

Suggested Files:

```
FoodItem.java
FoodBST.java
FoodAVLTree.java
SearchManager.java
DataStorage.java
```

Features to Demonstrate:

- Fast searching
- Sorted traversal
- Quick key-value retrieval

Concepts Covered:

- BST / AVL Tree
 - HashMap
 - Tree Traversal
 - Fast Lookup Systems
-

Shared Responsibilities (All Members)

System Integration

All members should work together to:

- Combine all modules into one system
- Create:

`Main.java`

- Build:
 - Menu system
 - Navigation flow
 - Testing integration
-

Suggested Project Structure

```
SmartFoodDeliverySystem/  
|  
├─ Main.java  
|  
├─ user_management/  
├─ order_processing/  
├─ delivery_assignment/  
├─ route_optimization/  
├─ search_retrieval/  
|  
├─ diagrams/  
├─ report/  
└─ README.md
```

Suggested GitHub Workflow

Each Member:

- Works on separate branch
- Pushes own module

Team Leader:

- Merges branches
 - Resolves conflicts
 - Final integration testing
-

Suggested Presentation Split

Member	Presentation Topic
Member 1	User & Restaurant Management
Member 2	Queue + Stack Order System
Member 3	Priority Queue Delivery Assignment
Member 4	Graph + Dijkstra Route Optimization
Member 5	BST/AVL + HashMap Retrieval

Recommended Java Data Structures

Requirement	Java Structure
Dynamic Storage	ArrayList / LinkedList
FIFO Processing	Queue
Undo Feature	Stack
Priority Selection	PriorityQueue
Route Finding	Graph
Fast Search	AVL Tree / BST
Fast Retrieval	HashMap

Final Deliverables

Code

- Complete Java project
- Working integrated system

Report

- Explanation of all data structures
- Complexity analysis
- Screenshots/output
- Diagrams:
 - Stack
 - Queue
 - Graph
 - Tree

Presentation

- Demonstration of each module
- Explanation of efficiency and design choices